Technical Design

Husna Keskin,
Jens Bouma,
Ben Wierenga,
Nicolas Prieto,
Matz Leeflang

Version Control

Version	Date	Description
0.1	01-10-2024	Initial document
0.2	22-10-2024	Updated C4-model
0.3	19-11-2024	Info for Api and Authentication Added.
0.4	28-11-2024	Info for managing routes and cohorts is added.
0.5	03-01-2025	Language corrections, changed c4-model and descriptions.
0.6	06-01-2025	Added new database diagram.
1.0	09-01-2025	Document ready

Table of Contents

1. Introduction 1

2. Techniques 2

2.1. JavaScript, TypeScript, Next.js 2

2.2. C#, ASP.NET 2

2.3. Postgresql 3

2.4. Docker 4

3. Context (Level 1) 4

3.1. Primary actors 5

3.2. Extern systems 6

4. Container (Level 2) 7

4.1. Communication 7

4.2. Keuzewijzer 8

4.3. Dashboard 8

4.4. REST API 8

5. Components (Level 3) 9

5.1. Keuzewijzer 9

5.2. Dashboard 10

5.3. REST API 11

5.4. Database 12

6. Code (Level 4) 13

6.1. Packages 14

6.2. Functionalities 15

7. Tests 19

7.1. Quality Auditor 19

7.2. Automatic tests 19

7.3. Manual tests 19

Figures 20

Bibliography 21

1. Introduction

This Technical Design document serves as a comprehensive guide for the development, architecture, and functionality of the Keuzewijzer system. It outlines the tools, technologies, and design principles used to create an interactive platform that supports Windesheim students and staff in exploring and managing graduation profiles, clusters, and cohorts. The document adopts the C4 model to present the system's architecture across multiple levels, from high-level context to detailed component and code-level insights.

The aim is to provide a clear understanding of the system's structure, components, and workflows. Each section builds upon the previous, offering a progressively detailed view of the system. Topics include the tools and frameworks employed (such as Next.js, ASP.NET, and SQLite), containerized deployment using Docker, and the interaction between primary actors, external systems, and internal components.

Versioning information captures the document's iterative development, while the technical details aim to assist developers, maintainers, and stakeholders in understanding and contributing to the project's success. Comprehensive testing strategies, including automated, manual, and quality audits, ensure the system's reliability, performance, and user experience. By adhering to these standards, this design strives to deliver a robust and user-centered solution.

2. Techniques

This chapter describes the different techniques we use for the database, programming languages and frameworks in this project.

2.1. JavaScript, TypeScript, Next.js

JavaScript (Docs, 2025), a versatile and high-level programming language, is crucial for web development alongside HTML and CSS. Primarily a client-side language, it operates directly in web browsers, enabling interactive web pages. Its key features include event-driven programming, allowing reactions to user actions; asynchronous programming for non-blocking operations like data retrieval, DOM manipulation to dynamically alter web content and structure, and prototypal inheritance, distinguishing it from classical object-oriented languages.

TypeScript (Microsoft T. , 2025), developed by Microsoft, is a strongly typed superset of JavaScript that compiles to JavaScript. It provides optional static typing to catch errors during development, making it ideal for large projects. Key features include static typing for error detection, support for modern ECMAScript features like classes and modules, and enhanced tooling for autocompletion, navigation, and refactoring.

Next.js (Vercel, 2025) is a React framework from Vercel that simplifies building fast, SEO-friendly web applications with features like Server-Side Rendering (SSR) for better load times and SEO, Static Site Generation (SSG) for faster page loads, integrated API routes for backend functionality, and built-in CSS and Sass support.

Together, these technologies provide a robust framework for building complex, scalable, and high-performance web applications. JavaScript forms the foundation of interactive web development, while TypeScript enhances this with type safety and better tooling, and Next.js offers advanced rendering capabilities and optimizations for React applications.

2.2. C#, ASP.NET

C# is a modern, object-oriented programming language developed by Microsoft, designed for building a wide range of applications. Known for its strong typing, performance, and versatility, C# supports paradigms like imperative, declarative, and functional programming. It integrates seamlessly with the .NET ecosystem, making it a go-to choice for enterprise-level applications, web development, and cloud services. Features such as LINQ, async/await, and an extensive standard library simplify development while enhancing productivity.

ASP.NET is a high-performance, open-source framework for building modern, scalable web applications using C#. It follows the Model-View-Controller (MVC) pattern and supports other architectures like Razor Pages and Web APIs. ASP.NET Core, the cross-platform version, is ideal for developing cloud-based, containerized applications. Key features include:

- **Dependency Injection**: Simplifies managing dependencies for scalable applications.
- **Routing**: Offers powerful URL routing for building RESTful APIs and clean URLs.
- **Middleware Pipeline**: Provides fine-grained control over request and response handling.
- **Integrated Security**: Supports authentication and authorization with built-in solutions like Identity and external providers (OAuth, OpenID Connect).

The ASP.NET framework offers robust tools for creating RESTful APIs, enabling seamless communication between client and server. Key components include:

- **Controllers**: Handle HTTP requests and return responses using JSON or XML.
- **Entity Framework (EF)**: A powerful ORM for working with databases through LINQ queries.
- **Model Binding and Validation**: Automatically maps and validates data from HTTP requests.
- **Authentication and Authorization**: Supports schemes like JWT, OAuth2, and custom policies.
- **Swagger/OpenAPI Integration**: Simplifies API documentation and testing.

2.3. Postgresql

PostgreSQL is a powerful, open-source relational database management system (RDBMS) known for its advanced features, reliability, and extensibility. It excels in handling complex queries, large datasets, and high-concurrency applications, making it a popular choice for scalable and performance-critical systems. PostgreSQL offers support for advanced data types (JSON, XML, arrays), full-text search, and geospatial data via PostGIS, making it

versatile for a wide range of use cases. Its ACID compliance ensures data consistency and integrity, critical for applications requiring robust transactional processing..

2.4. Docker

In the project, Docker (Docker, 2025) is utilized as a containerization platform that enables the efficient deployment and management of applications on the server. Docker allows developers to package applications and their dependencies into containers, which are lightweight, portable, and consistent across various environments. This means that applications can run seamlessly on any server that supports Docker, eliminating the common issues associated with environment discrepancies. Additionally, Docker simplifies the deployment process, allowing for quick updates and rollbacks, if necessary, thus enhancing the overall reliability and performance of the application running on the server.

3. Context (Level 1)

This chapter offers a high-level overview of the system's structure using the C4-model (C4-model, 2025), focusing on level 1, which outlines the system's context. It highlights the relationships between the system, users, external systems, and entities, identifying key stakeholders and their interactions. The chapter sets the foundation for a deeper exploration of the system's architecture.

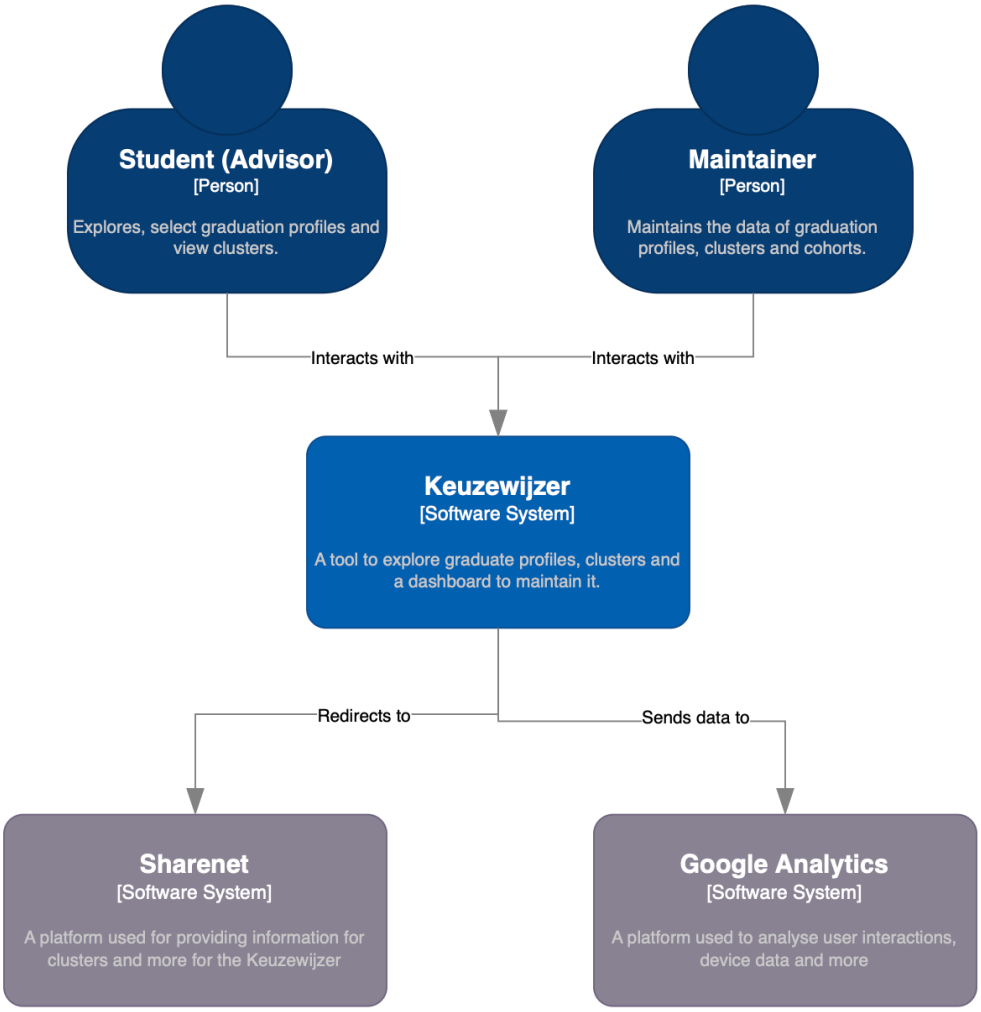


Figure 1 C4-model context level

3.1. Primary actors

3.1.1. Student

A Windesheim student uses the Keuzewijzer tool to explore and interact with available graduation profiles and clusters that can be pursued.

3.1.2. Student Advisor

A Windesheim student advisor uses the Keuzewijzer tool to guide and assist students in exploring graduation profiles and clusters, providing recommendations based on individual goals and progress.

3.1.3. Maintainer

A maintainer is responsible for overseeing and updating the Keuzewijzer tool, ensuring the accuracy and functionality of graduation profiles, clusters and other system features.

3.2. Extern systems

3.2.1. Sharenet

Sharenet or better Windesheim's SharePoint (Microsoft, 2025) platform, is used to provide detailed information on specific courses. Since the content of each minor is not displayed within the Keuzewijzer app, an information tab will be available for each course, redirecting users to the relevant webpage containing all the course details and information from the main Windesheim SharePoint platform.

3.2.2. Google Analytics

Google Analytics (Awati, 2024) will track user interactions within the Keuzewijzer tool, providing valuable data on how students engage with the system. By analyzing metrics such as page views, clicks, session duration, and user flow, it will help identify which features are most and least used, where students may face difficulties, and how they navigate through different graduation profiles.

This data-driven insight will enable the team behind the Keuzewijzer to make informed decisions about interface adjustments, content placement, or feature enhancements, ultimately improving the user experience and effectiveness of the tool in guiding students.

4. Container (Level 2)

This chapter delves into a more detailed explanation of the system's structure using the C4 model, focusing on level 2, which outlines the container view. It elaborates on the relationships and interactions between the different containers within the system, such as web applications, mobile apps, databases, and microservices. The chapter identifies how these containers collaborate to fulfill business requirements and sets the groundwork for understanding the system's operational architecture in greater depth.

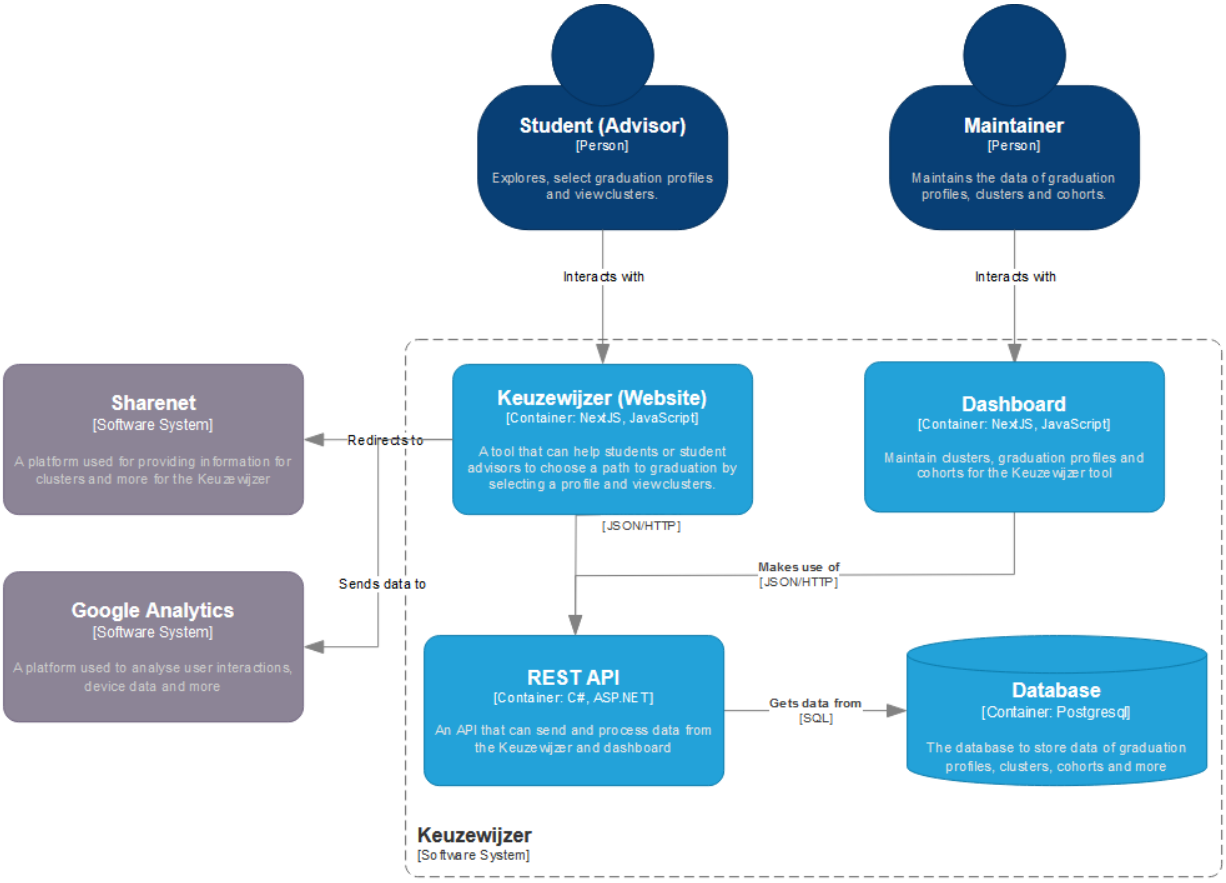


Figure 2 C4-model container level

4.1. Communication

The system uses JSON/HTTP requests to communicate between the REST API, Keuzewijzer (website), and the Dashboard. To obtain the most up-to-date data about descriptions, such as those for clusters, an external system is utilized. The REST API uses SQL queries to retrieve data from the database.

4.2. Keuzewijzer

The Keuzewijzer is a front-end application built with JavaScript and Next.js, designed for students. It displays available learning routes and their associated courses, allowing students to browse and select the best fit for their academic needs.

4.3. Dashboard

The dashboard is a front-end interface for maintainers that work at Windesheim. Built with JavaScript and Next.js, it provides tools for managing and updating the various graduate profiles, clusters and cohorts.

4.4. REST API

The REST API is built with ASP.NET to handle the communication and data exchange between the database, the Keuzewijzer, and the dashboard. This setup ensures seamless integration and efficient performance across all components of the system.

5. Components (Level 3)

This chapter delves into a more detailed explanation of the system's structure using the C4 model, focusing on Level 3, which provides a component view. It elaborates on the relationships and interactions between the different components within the containers. The chapter identifies how these components collaborate to fulfill specific functionalities within the containers and sets the groundwork for understanding the system's internal architecture in greater depth.

5.1. Keuzewijzer

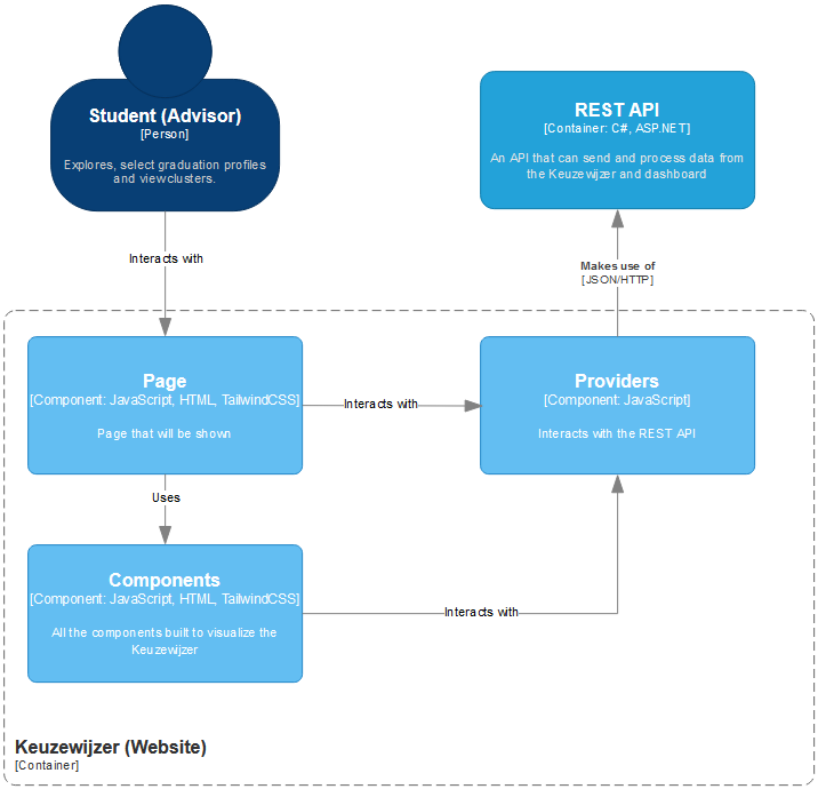


Figure 3 C4-model components level of Keuzewijzer (Website)

Keuzewijzer is built with Next.js using JavaScript for its flexibility and ease during rapid development cycles. JavaScript allows for swift prototyping and debugging, which is crucial for a dynamic, user-oriented project like Keuzewijzer. Its straightforward integration with Next.js ensures quick feature implementation and testing. The widespread popularity and community support of JavaScript also make it a dependable choice for creating interactive web applications, emphasizing speed and adaptability to user feedback.

5.2. Dashboard

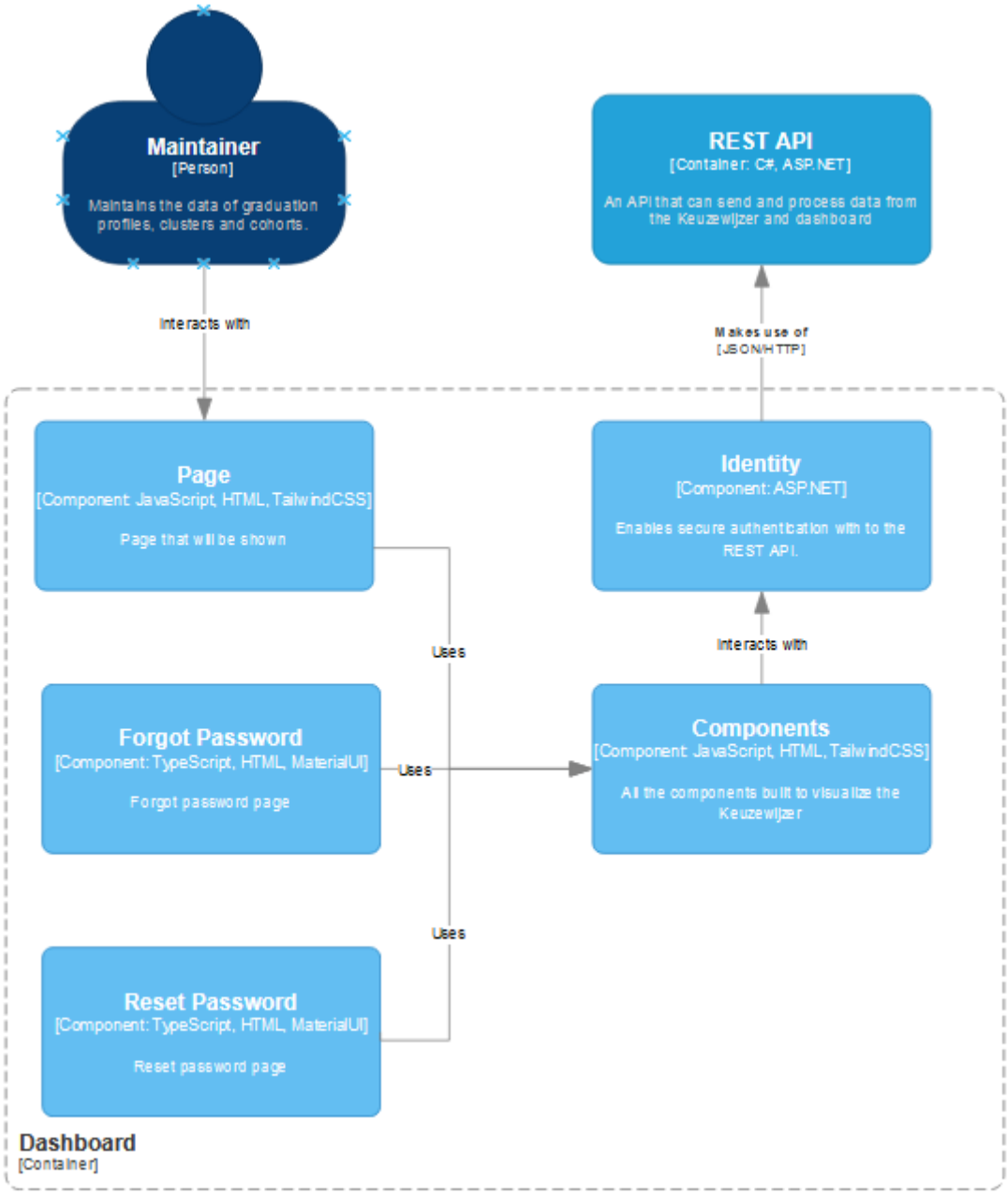


Figure 4 C4-model components level of the dashboard

The dashboard is developed using Next.js with TypeScript. When creating a dashboard, TypeScript is often preferred due to its strong typing system, which enhances code reliability and maintainability. TypeScript provides developers with early detection of errors during the development process, which can be incredibly beneficial for a project of this scale. Additionally, it offers excellent tooling support, making complex applications easier to

manage. As a result, the use of TypeScript in dashboard development not only improves the overall quality of the code but also streamlines the development workflow.

5.3. REST API

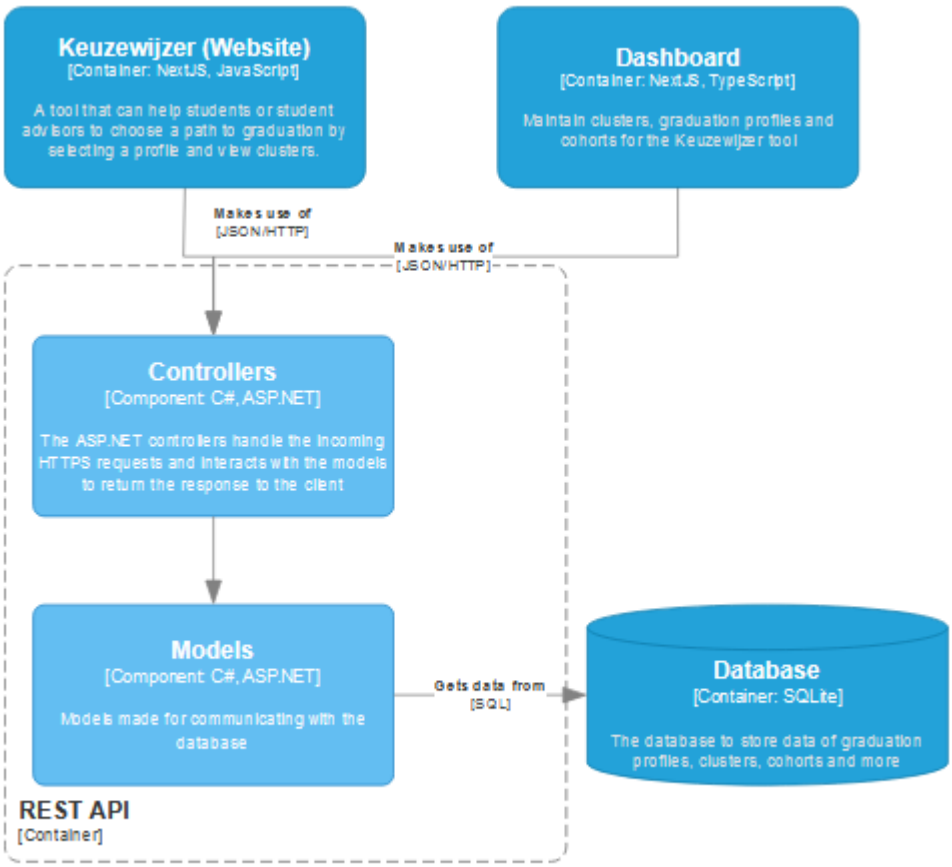


Figure 5 C4-model components level of the REST API

5.4. Database

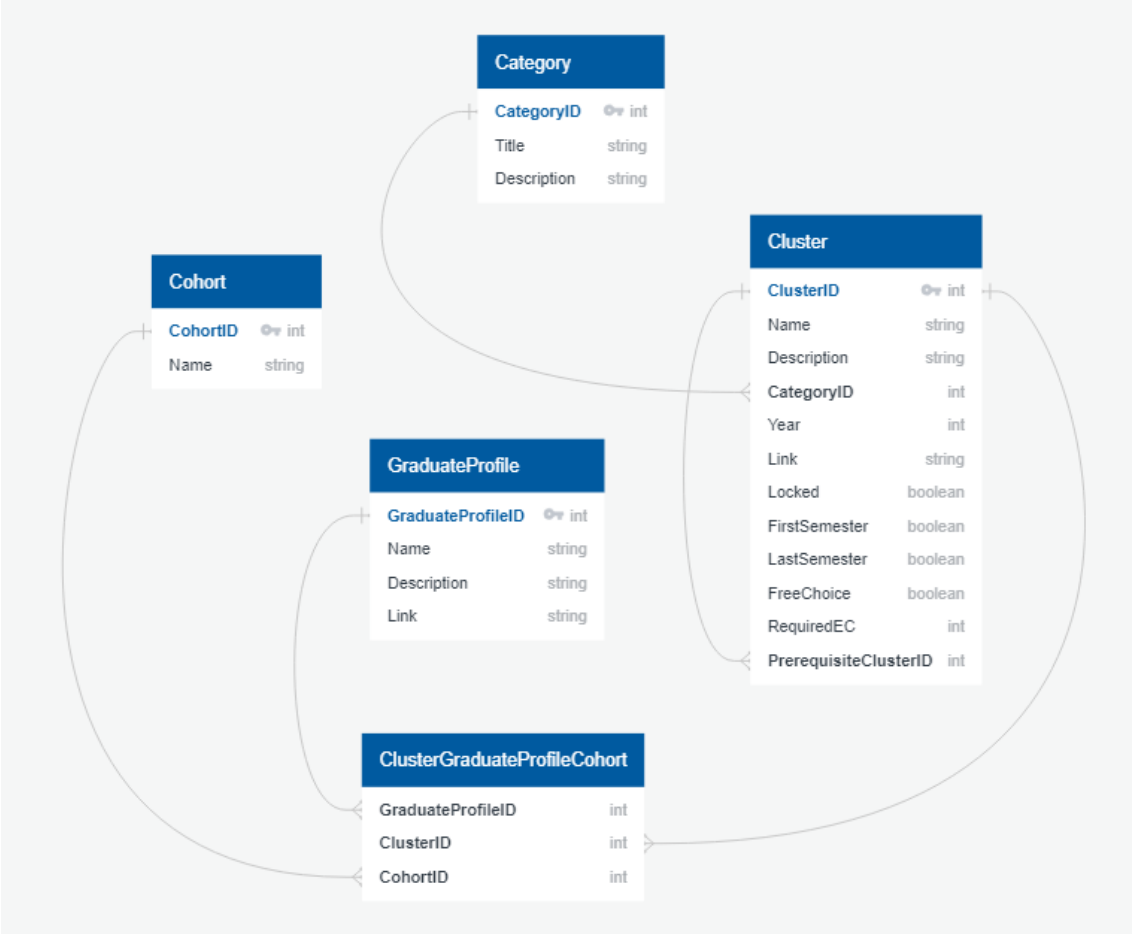


Figure 6 ERD of the Database

These tables illustrate the relationships between clusters and various domain elements. The primary relationship is shown in the ClusterGraduateProfileCohort table, which lists the clusters available for each cohort and graduate profile by including object IDs as foreign keys. Each cluster also belongs to a specific category, represented by a foreign key in the main Cluster table. Additionally, there is a one-to-one relationship within the cluster table itself, indicating that a cluster may have another cluster as a prerequisite.

Each table is derived from a model of the backend project and is constructed using ASP.NET to facilitate future migrations or structural changes without data loss.

6. Code (Level 4)

This chapter explores the system's architecture using the C4 model, focusing on Level 4, which provides a code-level view. It details the structure of classes, interfaces, and methods within the components. The chapter emphasizes the interactions and relationships between these elements, illustrating their collaboration to implement functionalities. It offers an in-depth look at the internal code structure, providing the knowledge necessary to comprehend the low-level design and behavior of the system.

Check the ERD to see the attributes of the models.

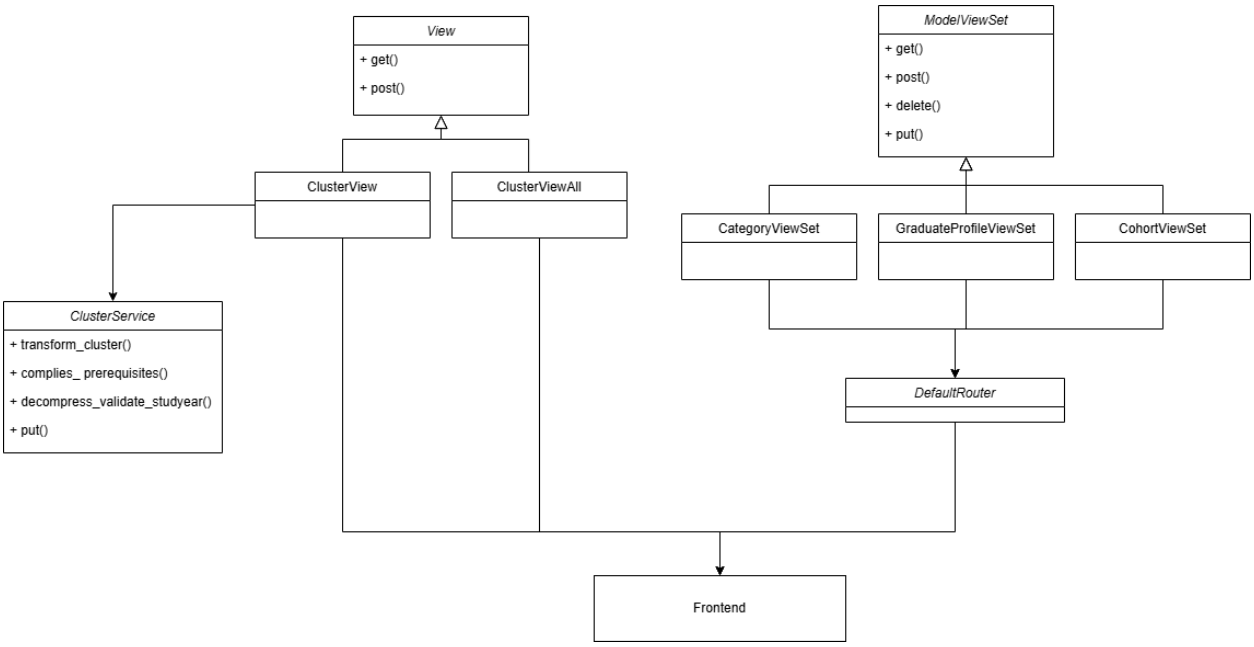


Figure 7 Class diagram of REST API Endpoints

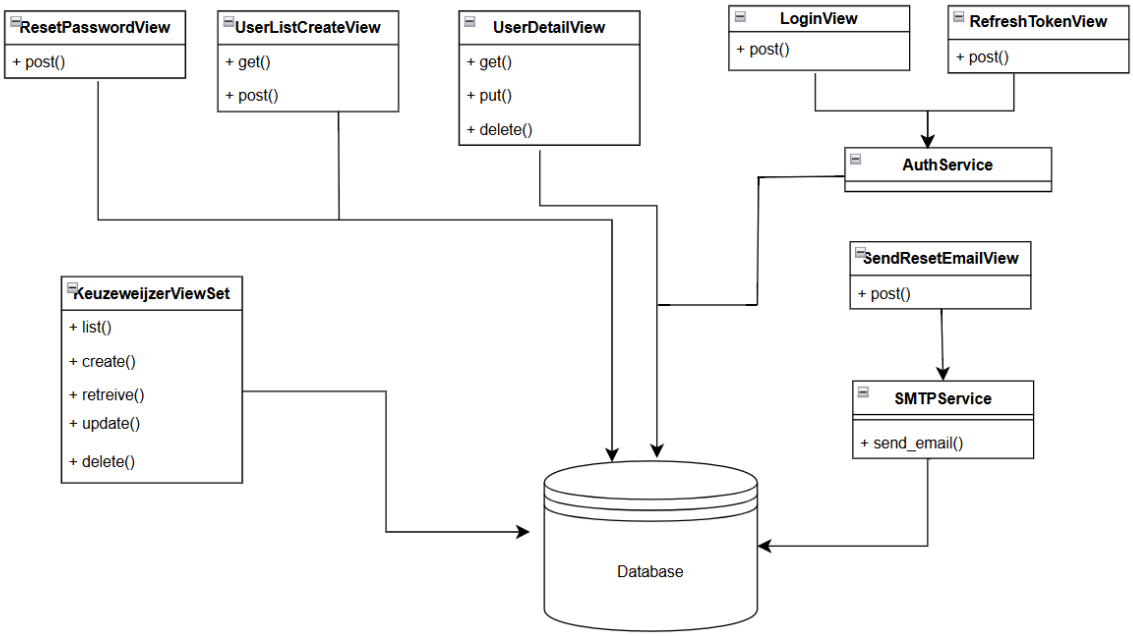


Figure 8 Class diagram of dashboard endpoints

6.1. Packages

6.1.1. Keuzewijzer

6.1.1.1. Axios

Axios (Axios, 2025) is a promise-based HTTP client for the browser and Node.js. It is designed to make it easier to send asynchronous HTTP requests to REST endpoints and perform CRUD (Create, Read, Update, Delete) operations. Axios simplifies tasks like handling responses, managing request headers, and supporting cross-site access control (CORS). It provides features like request cancellation, automatic transformation of JSON data, and handling of error messages. Additionally, Axios can intercept requests or responses before they are handled, making it a versatile tool for managing HTTP communication in web development.

6.1.1.2. TailwindCSS

TailwindCSS (Fitzgerald, 2022) is a utility-first framework that simplifies custom UI development by providing a wide range of low-level utility classes. It enables developers to design responsive, attractive web interfaces directly in HTML without custom CSS. Unlike traditional frameworks, it doesn't include ready-made components, offering a flexible base for creating unique designs. This boosts consistency, reusability, performance, and scalability across projects.

6.1.2. Dashboard

6.1.2.1. Axios

Axios (Axios, 2025) is a promise-based HTTP client for the browser and Node.js. It is designed to make it easier to send asynchronous HTTP requests to REST endpoints and perform CRUD (Create, Read, Update, Delete) operations. Axios simplifies tasks like handling responses, managing request headers, and supporting cross-site access control (CORS). It provides features like request cancellation, automatic transformation of JSON data, and handling of error messages. Additionally, Axios can intercept requests or responses before they are handled, making it a versatile tool for managing HTTP communication in web development.

6.1.3. REST API

6.1.3.1. Identity

ASP.NET Identity (Microsoft, 2025) is a library for managing user authentication and authorization in web applications and REST APIs. It provides tools to handle user accounts, roles, and claims, allowing secure access control. It integrates seamlessly with Entity Framework Core, supports token-based authentication (like JWT), and enables role-based and claims-based authorization. The system is customizable, letting you extend the user model and implement features like password recovery and multi-factor authentication.

6.2. Functionalities

6.2.1. Learning route creation

When the backend fetches clusters form the database, this returns a list of the clusters in an unstructured way. This list of clusters needs to be processed in a certain way in order to be sent to the backend to be properly displayed

In Order to do this, the backend creates a structure that will be referred to as “studyYear” This structure has the suitable format to be processed by the frontend and it consists of a series of nested arrays.

Each studyYear has a number of years, representing one academic year. Each Year contains two subsequent arrays “firstSemester” and “lastSemester” arrays each representing the two semesters that form a year. Each contains the list of clusters that can be taken on that semester. In order to dynamically build and display these studyYears the backend process the added clusters fitting each of them in the first available slot that outputs a still viable route.

Apart from these arrays each studyYear also contains a list of other variables used for logic purposes such as the studyLevels. These are indicators that are used both to check if a slot is completely free and to check if there is enough semester of the previous level before starting to add next level semesters to the route.

All logic processes that deal with the clusters provided from the database are managed from the ClusterService.py file where we define a series of static functions to be accessed from the rest of the backend.

- *transform_clusters:* this is the main function that takes a list of clusters as input and produces a studyYear to be sent to the frontend. This function only outputs functions that comply with the level conditions. In this function there are several "fixed" conditions, for example that the cluster taken on the first semester of the first year must always be "propedeuse 1". The process of cluster selection also forces afstuderen to be the last cluster on the route.

- *complies_prerequistites:* this function takes a studyYear structure and checks if the structure is valid as a finished route. It checks to see if there is any cluster whose prerequisites have not been fulfilled and if there is a level three cluster present in the route. If the route is properly built it will send a confirmation code to the frontend and with the study route. If not, depending on the error it will send a different code that indicates what is the problem with the current route.

- *retake_clusters:* This function is called whenever a cluster has the attribute retake different than zero. This means that the cluster is not only marked as chosen but has also been marked to be retaken. This function duplicates the cluster in the studyRoute as many times as the cluster has been marked to retake, and in the last position also adds the other possible alternatives to take.

6.2.2. Dashboard Authentication

The authentication system for the dashboard is implemented using NextAuth, a flexible authentication library for Next.js. It ensures that only authorized users can access protected routes and functionalities.

Middleware for Route Protection:

Middleware is used to intercept incoming requests and verify user authentication status. Protected routes are inaccessible to unauthenticated users, ensuring secure access to the dashboard.

Providers for API Communication:

Session Management:

Integration with REST API:

Authenticated requests to the REST API include user credentials (e.g., access tokens) provided by NextAuth, ensuring secure communication between the frontend and backend.

Security Measures:

Tokens are securely stored (e.g., HTTP-only cookies) to prevent client-side tampering. Middleware enforces role-based access controls, restricting functionalities based on user permissions.

API endpoints:

`POST /login/`: Checks the provided credentials and authenticate the user.

`POST /send-verification-code/`: Sends the verification code to the user's email.

`POST /verify-code/`: Validates the entered verification code.

`POST /reset-password/`: Resets the user's password after successful verification.

`POST /refresh/`: Refreshes the authentication token.

6.2.3. Managing graduate profiles, clusters and cohorts

This system allows maintainers to manage graduate profiles, clusters and cohorts and users via a dashboard. The dashboard interacts with the REST API to retrieve, update, and manage data related to graduate profiles, clusters and cohorts and users.

API endpoints:

1. **Graduate profiles**

`GET /graduate-profiles/`: Retrieve a list of graduate profiles.
`POST /graduate-profiles/`: Add a new graduate profile.
`PUT /graduate-profiles/{id}/`: Update an existing profile.
`DELETE /graduate-profiles/{id}/`: Delete a profile.

2. **Clusters**

`GET /clusters/`: Retrieve a list of clusters.
`POST /clusters/`: Add a new cluster.
`PUT /clusters/{id}/`: Update an existing cluster.
`DELETE /clusters/{id}/`: Delete a cluster.

3. **Cohorts**:

`GET /cohorts/`: Retrieve a list of cohorts.
`POST /cohorts/`: Add a new cohort.
`PUT /cohorts/{id}/`: Update an existing cohort.
`DELETE /cohorts/{id}/`: Delete a cohort.

4. **Users:**

`GET /users/`: Retrieve a list of users.
`POST /users/`: Add a new user.
`GET/users/{id}/`: Retrieve details of a specific user.
`DELETE /users/{id}/`: Delete a user.
`PUT /users/{id}/`: Update details of the user.

7. Tests

7.1. Quality Auditor

The system will be tested with Lighthouse (Google, 2016) to measure its performance, accessibility, and Search Engine Optimization (SEO). This ensures that the system meets modern web standards, provides an inclusive user experience, and is optimized for search engines.

7.2. Automatic tests

The system will be automatically tested using Cypress (Cypress, 2025). Cypress can test the application from end-to-end (E2E). Which ensures that running in a browser, functionality can be tested from the frontend and backend at once.

Using Cypress as automatic tests can save time unlike manual tests. For features that cannot be tested E2E, unit tests will be made.

7.3. Manual tests

Some parts of the system cannot be tested automatically. To still test these parts of the system, manual tests will be done by a tester. All tests that cannot be done automatically are visible in a test report document.

Figures

Figure 1 C4-model context level 5
Figure 2 C4-model container level 7
Figure 3 C4-model components level of Keuzewijzer (Website) 9
Figure 4 C4-model components level of the dashboard 10
Figure 5 C4-model components level of the REST API 11
Figure 6 ERD of the Database 12
Figure 7 Class diagram of REST API Endpoints 13
Figure 8 Class diagram of dashboard endpoints 14

ASP

Bibliography

Awati, R. (2024, April). *Google Analytics*. Retrieved from TechTarget: https://www.techtarget.com/searchbusinessanalytics/definition/Google-Analytics

Axios. (2025). *Getting started*. Retrieved from Axios: https://axios-http.com/docs/intro

C4-model. (2025). *C4-model*. Retrieved from Introduction: https://c4model.com/introduction

Cypress. (2025). *Cypress home page*. Retrieved from Cypress: https://www.cypress.io

Django. (2025). *Django*. Retrieved from Django: https://www.djangoproject.com

django-rest-framework. (2025). *Django REST framework*. Retrieved from django-rest-framework: https://www.django-rest-framework.org

Docker. (2025). *Docker*. Retrieved from What is Docker?: https://docs.docker.com/get-started/docker-overview/

Docs, M. W. (2025). *JavaScript*. Retrieved from Modzilla: https://www.typescriptlang.org

Fitzgerald, A. (2022, May 31). *Tailwind CSS: What is it, why use it & examples*. Retrieved from HutSpot: https://blog.hubspot.com/website/what-is-tailwind-css

Google. (2016, Sept 27). *Introduction to Lighthouse*. Retrieved from Google: https://developer.chrome.com/docs/lighthouse/overview

Gunicorn. (2025). *Gunicorn home page*. Retrieved from Gunicorn: https://gunicorn.org

MaterialUI. (2025). *MaterialUI - Overview*. Retrieved from Material UI: https://mui.com/material-ui/getting-started/?srsltid=AfmBOop5CNpl9uK4yczyPXa2kSsyB5d0YsgZXMgxqD30NOBjE-SBAde3

Microsoft. (2025, 01 13). *Introduction to ASP.NET identity*. Retrieved from Microsoft: https://learn.microsoft.com/en-us/aspnet/identity/overview/getting-started/introduction-to-aspnet-identity

Microsoft. (2025). *What is SharePoint?* Retrieved from Microsoft: https://support.microsoft.com/en-us/office/what-is-sharepoint-97b915e6-651b-43b2-827d-fb25777f446f

Microsoft, T. (2025). *TypeScript*. Retrieved from TypeScript: https://www.typescriptlang.org

NextAuth. (2025). *Introduction*. Retrieved from NextAuth: https://next-auth.js.org/getting-started/introduction

Pypi. (2024, March). *Asgiref project description*. Retrieved from Pypi: https://pypi.org/project/asgiref/

Pypi. (2024, Oct 29). *django-cors-headers project description*. Retrieved from pypi: https://pypi.org/project/django-cors-headers/#:~:text=django%2Dcors%2Dheaders%20is%20a,Origin%20Resource%20Sharing%20(CORS).

Pypi. (2024, Jan 23). *python-dotenv project description*. Retrieved from Pypi: https://pypi.org/project/python-dotenv/

Python. (2025). *What is Python? Executive summary*. Retrieved from Python: https://www.python.org/doc/essays/blurb/

SQLite. (2025). *SQLite*. Retrieved from SQLite: https://www.sqlite.org/

Vercel. (2025). *Introduction*. Retrieved from Next.js: https://nextjs.org/docs